Why Ultimate Tic-Tac-Toe Is a Surprisingly Hard Game

You probably learned tic-tac-toe as a child and stopped finding it interesting around age seven. Two players take turns marking a 3×3 grid; get three in a row and you win. With a bit of thought, any competent player can force a draw every time. The game is, in the technical sense, solved — there is nothing left to discover.

Ultimate Tic-Tac-Toe takes this children's game and does something to it that should feel modest but turns out to be profound. It asks: what if each cell of the tic-tac-toe board were itself a tic-tac-toe board?

The Rules

The board is a 3×3 grid of 3×3 grids — nine sub-boards arranged into one meta-board, 81 cells in total. To win, you need to win three sub-boards in a row on the meta-board. Winning a sub-board works the same as ordinary tic-tac-toe: get three of your pieces in a row within that 3×3 grid.

That's the easy part. Here's the twist.

When you place a piece in cell k of a sub-board, your opponent must make their next move in sub-board k. The cell you play in dictates where your opponent is sent.

A concrete example. Say it's the opening move and you play in the top-left sub-board, choosing the center cell (cell 4 within that sub-board). Your opponent is now forced to play in sub-

board 4 — the center sub-board of the meta-board. They play in the bottom-right corner of that sub-board (cell 8). You are now sent to sub-board 8 — the bottom-right of the meta-board. And so on.

There is one exception: if you are sent to a sub-board that has already been won or drawn, you get free choice — you may play anywhere on any unfinished sub-board.

Two things follow immediately from this mechanic. First, winning a sub-board doesn't just score a point — it sends your opponent to a specific sub-board of your choice (whichever cell you used to secure the win). Second, *you sometimes want to avoid winning a sub-board*, because winning it in the wrong cell would give your opponent a free ride to a sub-board that benefits them.

This is the first hint that something unusual is going on strategically.

How Complex Is This, Actually?

To understand why this game is hard, it helps to place it on a spectrum of game complexity.

Tic-tac-toe. Roughly 9! = 362,880 possible game sequences (an overcount, since games end early). The full game tree fits comfortably in memory. A computer can enumerate every possible game in milliseconds. Optimal play is a draw, always — and once you know the strategy, you'll never lose.

Chess. Claude Shannon estimated roughly 10^120 possible games — a number so large it dwarfs the number of atoms in the observable universe (~10^80). No computer will ever enumerate the full tree. Instead, chess engines use *evaluation functions* — hand-crafted heuristics that score a position by

counting material (a queen is worth more than a pawn), assessing king safety, controlling the center, and dozens of other features. These functions are approximate but good enough that search algorithms can use them to play at superhuman levels.

Go. Roughly 10^360 possible games. For decades, the dominant wisdom was that Go was too complex for computers because the board was too large and evaluation functions too unreliable — the game's emergent patterns of territory and influence resist reduction to simple heuristics. This turned out to be correct, and it took a fundamentally different approach (AlphaGo, 2016) to finally crack it.

Ultimate Tic-Tac-Toe. Where does it sit? Let's think through the game tree.

At the very first move, all 81 cells are available. After the first move, the current player is sent to a specific sub-board with typically 9 empty cells. So the branching factor drops immediately from 81 to 9, and for much of the game it stays near 9. As sub-boards fill up or get won, the branching factor fluctuates — sometimes you're sent to a nearly-full board and have only 2 or 3 options; occasionally you get free choice across many sub-boards.

A typical game lasts 30–60 moves. Taking a typical branching factor of ~9 — after the opening move you are almost always sent to a specific sub-board with around 9 empty cells — the game tree has on the order of 9^40 ≈ 10^38 possible games. That's far smaller than chess (10^120) and astronomically smaller than Go (10^360).

It's worth separating two things that often get conflated here. The *game tree* — the number of distinct ways a game can be played out — is what 9^40 estimates: roughly 10^38 possible sequences of moves. Separately, the *state space* — the number

of distinct board configurations that can appear — is bounded by 3^81 ≈ 4.4×10^38, since each of the 81 cells can be empty, X, or O. These happen to be similar in magnitude, but they measure different things. The game tree tells you how hard search is; the state space tells you how much memory a lookup table would need.

So Ultimate TTT should be *easier* than chess to solve computationally, right?

Not quite. Raw state count isn't the whole story. The harder question is: *can you build a good evaluation function for it?*

Why Classical Chess Engines Break Down Here

Chess engines work by combining two things: a search algorithm (typically minimax with alpha-beta pruning) and an evaluation function. The search explores the game tree up to some depth, and the evaluation function scores positions at the leaves of the search. The idea is that you don't need to search to the end of the game — you just need to search far enough that a decent heuristic can distinguish good positions from bad ones.

This works in chess because "more pieces = better" is approximately true. A position where you're up a rook is, all else being equal, a winning position. The evaluation function has real signal to work with. It's imperfect, but it's good enough that deeper search reliably finds better moves.

In Ultimate TTT, there is no analogous local heuristic. Consider: what does it mean to have won 5 out of 9 sub-boards? It sounds good. But if those 5 wins are scattered randomly across the meta-board, with no three in a line, and your opponent holds the three sub-boards in the top row, you are losing. Worse, those 5 won sub-boards are now *locked* — you can't play in them —

which means if your opponent sends you there, you get free choice, which is actually an advantage for you. The relationship between "winning more sub-boards" and "being in a better position" is non-monotonic and highly dependent on global board structure.

More subtly: *winning a sub-board might be a strategic mistake*. Suppose you could win sub-board 4 (the center) by playing in cell 0 of that sub-board. Cell 0 sends your opponent to sub-board 0 — the top-left corner. If your opponent is building a strong position in sub-board 0, you'd be handing them exactly what they want. It might be better to not win sub-board 4 yet, and instead play a different cell that sends your opponent somewhere less useful.

This coupling between the cell you play, the sub-board you send your opponent to, and the global meta-board consequence is what makes depth-limited search with a heuristic evaluation essentially useless. The "value" of a local move depends entirely on global context that a simple feature-based function can't capture. You'd need to search much deeper to see the consequences, but deep enough search is computationally infeasible given the branching factor.

Chess evaluation functions took decades of expert knowledge to build. For Ultimate TTT, no one has managed to construct one that reliably works — the non-local dependencies are simply too entangled.

Why the AlphaGo Approach Works

The insight that made AlphaGo possible, and that applies equally here, is to stop trying to hand-craft an evaluation function and instead *learn one from data*.

If you play millions of games against yourself, starting from random play and gradually improving, you accumulate an enormous dataset of positions and outcomes. From this data, a neural network can learn: "from positions that look like this, the current player tends to win." Not because anyone told it what features matter, but because the statistical patterns in millions of games contain that information implicitly.

This is the value network — a function that takes a board position and outputs a number between −1 (current player loses) and +1 (current player wins). It's not computing material counts or positional features. It's doing something much harder to interpret but much more powerful: it's learned a compressed representation of what winning positions look like, drawn from the actual distribution of games.

Paired with this is the policy network — a function that, given a board position, outputs a probability distribution over legal moves, representing which moves are worth exploring. Together they power Monte Carlo Tree Search (MCTS): the policy network focuses search on promising moves, and the value network evaluates positions without searching to the terminal state.

The beauty of this approach for Ultimate TTT specifically is that you don't need to understand *why* the sending mechanic creates long-range dependencies. The network discovers this from self-play. It will learn that winning certain sub-boards is bad, that certain meta-board configurations are strategically decisive, and that the apparent local value of a move can be inverted by global considerations — all without being told any of this.

What you do need is a way to represent the board state as an input to a neural network, a training procedure that generates

good self-play data, and an architecture powerful enough to capture the non-local dependencies that make the game hard.

What you need, then, is a system that learns from playing games — not a hand-crafted function that someone wrote down, but something that accumulates experience and gradually figures out which decisions tend to lead to wins. Watching millions of games and asking: given everything that happened, what did good play actually look like? That question is harder than it sounds, and answering it well turns out to require a surprisingly careful set of ideas.

The Credit-Assignment Problem: A Tour of Reinforcement Learning

The last essay explained why Ultimate Tic-Tac-Toe resists classical engines: the sending mechanic creates non-local dependencies that no hand-crafted evaluation function has reliably captured. The solution is to learn one from self-play. But we haven't explained *how* — what the learning algorithm is doing, why some approaches fail, and why the one we actually use (MCTS + neural networks) succeeds.

That's what this essay is about. It covers the landscape of reinforcement learning algorithms, not exhaustively, but enough to understand the key tension each one is wrestling with. Every method below is trying to solve the same underlying problem. Watch how each one localises the learning signal.

---

1. The Credit-Assignment Problem

You play a 40-move game. You win. You receive reward +1.

Move 7 was a blunder — you voluntarily gave your opponent a strong sub-board. You survived it only because they misplayed several moves later. Move 23 was the key move: you sent your opponent to a sub-board where any response would be bad for them, and that constraint propagated through the rest of the game and secured your win.

Now: how does the learning algorithm know the difference?

It doesn't. At least not obviously. All it has is a single number — +1 — arriving at the end of the game, and a transcript of 40

decisions that led there. The algorithm has to figure out, somehow, that some of those decisions were good and some were bad. This is the credit-assignment problem, and it is arguably the central difficulty in reinforcement learning.

The problem has two dimensions. The *temporal* dimension: the consequences of a decision often arrive many steps later. If you make a bad structural move in the opening, you may not feel the consequences until the endgame. The *counterfactual* dimension: to know whether move 23 was good, you'd need to know what would have happened if you'd played something else — and you don't, because you only played the game once.

Every algorithm in this essay is a partial answer to this problem. None of them solve it completely. What matters is how cleanly each one localises the signal.

---

2. Value Functions and the Bellman Equation

Before we can talk about algorithms, we need a language for describing "how good is this position."

Define **V(s)** as the expected total future reward when starting from state s and playing optimally from there. This is the *state-value function*. Define **Q(s, a)** as the expected total future reward when starting from state s, taking action a, and playing optimally thereafter. This is the *action-value function*. The difference is whether you've already committed to an action or not.

Both of these are notional — in a complex game, you can't compute them directly because that would require simulating every possible future. But the Bellman equation gives you a way to express them *recursively*, which turns out to be tractable:

```
V(s) = r(s, a) + γ · V(s')
```

Read this as: the value of being in state *s* equals the immediate reward you get for taking action *a*, plus a discounted version of the value of the state you land in. The discount factor γ (between 0 and 1) makes near-term rewards count more than distant ones.

This recursion isn't just a convenient definition — it follows from the law of total expectation (also called the tower property). The value of a state is defined as the expected total discounted return: V(s) = E[G_t | s_t = s], where G_t = r_t + γr_{t+1} + γ²r_{t+2} + …. Expanding one step: E[G_t] = E[r_t + γG_{t+1}]. By the tower property, E[γG_{t+1}] = γ·E[E[G_{t+1} | s_{t+1}]] = γ·E[V(s_{t+1})]. So V(s) = E[r_t + γ·V(s_{t+1})], which is exactly the Bellman equation. The recursive structure isn't an assumption — it's a direct consequence of conditioning on the next state.

The equation above — V(s) = E[r + γ·V(s')] — describes how to evaluate a *fixed* policy π. But the deeper question is: what is the *best* value you can achieve from any state, across all possible policies? This turns out to arise from an optimisation problem.

Define the objective as finding the policy π* that maximises expected discounted return from any starting state:

```
V*(s) = max_π E_π[ Σ_{t=0}^∞ γ^t · r_t | s_0 = s ]
```

Bellman's **principle of optimality** states that an optimal policy, viewed from any intermediate state, must remain optimal from that state onward — you can't have a globally optimal policy that makes a locally suboptimal choice at any point. This recursive

structure forces V* to satisfy what is called the **Bellman optimality equation**:

```
V*(s) = max_a [ r(s, a) + γ · E[V*(s')] ]
```

This looks like the policy-evaluation Bellman equation but with a max over actions instead of an expectation under a fixed policy. It's not a definition — it is a necessary condition for optimality derived from that recursive argument.

The discount factor $\gamma < 1$ does two things that are often conflated. The first is intuitive: near-term rewards count more than distant ones. The second is mathematical: $\gamma < 1$ makes the infinite sum converge (since $|\Sigma\ \gamma^t\ r_t| \le r_{max} \cdot 1/(1-\gamma) < \infty$ for bounded rewards), and it makes the Bellman operator T^* a **γ-contraction** in the sup-norm:

$$\|T^*V_1 - T^*V_2\|_\infty \le \gamma \cdot \|V_1 - V_2\|_\infty$$

By the **Banach fixed-point theorem**, a contraction on a complete metric space has a unique fixed point — and that fixed point is V^*. The practical consequence is value iteration: start from any initial estimate V_0, repeatedly apply T^*, and the sequence V_0, T^*V_0, $T^{*2}V_0$, … converges to V^* regardless of where you started. This is the theoretical foundation for why RL works: the optimal value function exists, is unique, and is reachable by iteration. The neural network in AlphaZero is approximating this fixed point.

It's worth being precise about what r actually is, because it's easy to conflate things that the algorithm treats very differently. r(s, a) is the *reward function* — defined by the game rules, not learned. It's handed to the algorithm by the environment. For Ultimate TTT, that means r = 0 on every non-terminal move, r = +1 for a move that wins the game, r = −1 for a move that loses

it, and r = −0.5 for a draw (the last value is a design choice; we return to it later). What the algorithm *learns* is V(s) and Q(s, a) — the predicted accumulated future reward from each position. The Bellman equation makes this distinction crisp: r is the known ground-truth signal arriving from the game; V is the unknown we're solving for. You can think of r as the measurement and V as the model trying to predict the integral of all future measurements.

This recursion is the engine of RL. Every algorithm below is essentially a different strategy for using this equation to bootstrap better estimates from worse ones.

---

3. Two Design Choices

Before diving into specific algorithms, it helps to establish the two independent choices every RL method makes.

Choice 1: What are you estimating?

The first is what you are trying to learn. You could estimate V(s) — for each state, a single number representing the expected future return from that position. This is a *vector*: one entry per state, indexed by state. State 47 → 0.6. State 48 → −0.2. Cheap to store, but it doesn't directly tell you what to do — to act, you'd need to know what state each action leads to.

Alternatively, you could estimate Q(s, a) — for each (state, action) pair, the expected return if you take that action from that state. This is a *matrix*: rows are states, columns are actions, and each cell holds one number. State 47, action 3 → 0.4. This is the actual "table" in tabular Q-learning. It's more expensive (|S|×|A| entries vs |S| entries for V), but it gives you the policy directly: from any state, just take the action with the highest Q value — no model of transitions needed.

For Ultimate TTT with 3^81 ≈ 4.4×10^38 possible board configurations (each of the 81 cells can be empty, X, or O — 3^81 possible configurations) and ~9 legal actions per state, neither is storable. But the distinction between V and Q matters for what neural networks will later be asked to approximate.

Choice 2: How much of the game do you observe before updating?

This is the dimension most textbooks obscure by presenting methods in sequence rather than as a spectrum.

One strategy: play the game all the way to the end, observe the actual outcome, then go back and update every state you visited. This is *Monte Carlo*. You wait for the ground truth and update from it directly.

The other strategy: update after every single step, using your current estimates to fill in what you expect to happen next. This is *Temporal Difference (TD)* learning. You don't wait for the game to end — you bootstrap off your own predictions.

These are genuinely different strategies, not just labelling choices. Monte Carlo updates are unbiased — you're using the actual return — but high-variance, because a single game is noisy. TD updates are lower-variance but introduce bias, because you're bootstrapping off estimates that may be wrong. Everything that follows is a variation on this tradeoff.

Placing both dimensions on a grid:

	V(s)	**Q(s,a)**
Monte Carlo update	MC V-learning	MC Q-learning
TD update	TD V-learning	Q-learning

"Q-learning" — which gets its own section — is TD applied to Q rather than V, with a max over next actions to handle the off-policy case. It is not a third independent idea; it sits in the TD + Q cell. AlphaZero sits in the MC + V cell: game outcomes train the value head, one full game at a time. Notice also that REINFORCE (covered below) is the MC approach applied to policy learning, and actor-critic is the TD approach applied to policy learning. The same dimension runs through the whole essay.

---

4. Monte Carlo: Play the Whole Game, Then Update

The most direct approach: play a complete game, observe the outcome, and then update every state you visited.

Formally, define the *return* from step t as:

```
G_t = r_t + γ·r_{t+1} + γ²·r_{t+2} + ... + γ^{T-t}·r_T
```

This is the total discounted reward from step t to the end of the game at step T. After the game finishes, update each state's value estimate toward the actual return you observed:

```
V(s_t) ← V(s_t) + α·(G_t − V(s_t))
```

Move the estimate toward what actually happened. α is the learning rate.

For Ultimate TTT, where r=0 on all intermediate moves and r=±1 or r=−0.5 at the terminal, G_t = γ^{T-t}·r_T for all non-terminal steps. The same discounted game outcome updates every position in the trajectory.

Monte Carlo has a clean property: the estimates are *unbiased*. You're directly observing the actual return — no approximation. The cost is variance: a single game is noisy, so you need many games to average out the randomness. And you must wait for the game to end before making any update. For a 40-move game, that means 40 moves of delay before the first gradient step.

The deeper problem is credit assignment: the same G_t multiplies the update for every state in the game, regardless of whether any individual decision was particularly good or bad. The win or loss gets smeared evenly across all 40 positions. This is correct in expectation but slow in practice — it takes many games for the estimate of a specific position to converge, because each game's outcome is a noisy signal about that one position specifically.

---

5. TD Learning: The Update Rule

The most direct application of the Bellman insight is *temporal difference (TD) learning*. Rather than waiting for the game to end to figure out what happened, update your value estimates after every single step.

After observing a transition — you were in state *s_t*, took action *a_t*, received reward *r_t*, and ended up in state *s_{t+1}* — compute the **TD error**:

```
δ_t = r_t + γ · V(s_{t+1}) − V(s_t)
```

Then update:

```
V(s_t) ← V(s_t) + α · δ_t
```

The TD error is the difference between what your current estimate predicted and what the Bellman equation says you should have predicted, given the next state. If $\delta_t > 0$, this step turned out better than expected; if $\delta_t < 0$, it turned out worse. α is the learning rate — a small step size that controls how aggressively you revise.

There's a subtlety worth making explicit for board games. In Ultimate TTT, the only non-zero reward is at the terminal state: +1 for a win, -1 for a loss, 0 for a draw. Every intermediate step yields $r_t = 0$. So for the first 39 moves of a 40-move game, the update reduces to pure bootstrapping:

$$V(s_t) \leftarrow V(s_t) + \alpha \cdot [0 + \gamma \cdot V(s_{t+1}) - V(s_t)]$$

No real reward signal appears in this update — just your current estimate of the next state being used to nudge your current estimate of this one. The actual signal only enters at the terminal step, when the game result arrives and V(terminal) is pinned to +1 or -1. From there, the correction propagates backward across subsequent games: the terminal state gets corrected first, then the second-to-last position in the next game it appears in, then the position before that, and so on. The wave of corrections spreads backward one step per episode. TD doesn't avoid playing the game many times — it amortises the correction across many small updates over many games rather than applying it all at once.

But this "correction wave" description is misleading in a way that matters. It implies the signal eventually reaches early positions if you wait long enough. For Ultimate TTT, it doesn't.

The wave argument assumes you'll revisit s_{T-2} *after* $V(s_{T-1})$ has been updated — so that the next time you transition $s_{T-2} \to s_{T-1}$, the TD error is nonzero and $V(s_{T-2})$ can learn. But the state space has roughly 3^{81}

positions. Every game traces a nearly unique path through it. Game 1 ends at some terminal T_1 and updates V(s_{T-1}^{(1)}). Game 2 ends at T_2 and updates V(s_{T-1}^{(2)}). These are almost certainly different states. After a million games you've touched roughly a million entries in a vector of 3^81 — each at most once, each only at the final step. The probability that any future game passes through one of those exact positions again is essentially zero.

The correction never propagates. Every position earlier than the last move remains at V = 0 indefinitely. The algorithm is running but learning nothing about non-terminal positions. This isn't a matter of needing more games — it's a structural impossibility. You would need to play a number of games comparable to the size of the state space before any meaningful coverage develops, and 3^81 is not a number any computer will reach.

This is why neural networks are not just a convenient approximation for large games — they are the only mechanism by which learning is possible at all. A network with shared parameters *generalises*: updating the estimate for state s also adjusts estimates for similar states s′, because they share weights. The exact board configuration never needs to be revisited. Signal spreads across the state space through the network's inductive bias, not through repeated state visitation.

That generalisation is the escape hatch. But it introduces a new problem: the value estimates are now only as good as the network's ability to generalise, which depends on having learned useful representations from the games seen so far. Early in training, the network generalises poorly. We'll return to this when we discuss why actor-critic methods have a quality ceiling.

Compare this to *Monte Carlo* methods, which wait for the episode to end and then update every state in that game simultaneously with the same return G_t. The MC approach sees

the full picture immediately but only after the game is over; TD sees partial corrections immediately but spreads the work across many games. Both approaches eventually converge to the same answer; they differ in how quickly corrections propagate and how much variance those corrections carry.

One more thing worth flagging now, since it will matter later: in the AlphaZero setup we're actually building, the value target is Monte Carlo-style. We play a full game, observe the outcome z, and label every position in that game with z as the regression target. The TD-like structure doesn't appear in the outer training loop — it appears *inside MCTS itself*. During the backup step, the leaf value V(s_leaf) is propagated back up through the search tree, negated at each level to account for the alternating-player structure. That's a TD-style one-step bootstrap through the search tree, not through real game steps. The distinction will be cleaner once we see exactly how MCTS works.

This is still credit assignment by imputation, not direct measurement. But the locality is meaningfully better than waiting for the terminal state and updating all 40 moves in one shot.

---

6. Q-Learning: TD Applied to Q

TD V-learning updates V(s) — state values. But a policy needs to know not just *where* to be, but *what to do*. The natural extension is to apply the same TD update rule to Q(s, a) instead — the value of taking a specific action from a specific state.

As established in the taxonomy above: Q(s, a) is the target function, TD is still the mechanism. The update looks almost identical, but the one-step lookahead now takes the best available action in the next state:

```
Q(s, a) ← Q(s, a) + α · [r + γ · max_{a'} Q(s', a') − Q(s, a)]
```

Update your estimate of Q(s, a) toward the reward actually received plus the best value you can get from the next state. The max operator is what makes this *off-policy*: you update toward the best possible next action regardless of which action you actually took, which lets you learn the optimal policy even while following an exploratory one. Once you have Q, the policy is trivial: from any state, take the action with the highest Q value.

This works beautifully in small problems. The catch for Ultimate TTT: there are roughly 3^81 ≈ 4.4×10^38 distinct board configurations. A lookup table mapping states to Q values is computationally impossible. You can't store it, can't index into it, and can't traverse it. The algorithm is correct in principle and completely useless in practice at this scale. This is the *scaling wall* that tabular RL hits in any interesting game.

---

7. DQN: Approximate with a Neural Network

The obvious response to the scaling wall: replace the table with a neural network. Instead of Q(s, a) as a lookup, use Q_θ(s, a) where θ are the parameters of a network. The network generalises — it can assign Q values to states it has never seen, by pattern-matching to similar states it has.

This is Deep Q-Networks (DQN), the algorithm that achieved human-level performance on Atari games in 2013. For a game like Pong or Breakout, it works extremely well: the screen is the state, there are a small number of joystick actions, and the network can learn useful features from pixel patterns.

Getting DQN to train stably requires solving a subtle problem. The natural loss function is:

```
L(θ) = E[ (r + γ · max_{a'} Q_θ(s', a') − Q_θ(s, a))² ]
```

Minimise the squared difference between what Q predicts now and what the Bellman equation says it should predict. But notice that θ appears on both sides. Every gradient step updates Q_θ(s, a) — the prediction — and simultaneously shifts r + γ · max Q_θ(s', a') — the target. You're chasing a moving target with a gun that moves in the same direction. In practice this causes the training to oscillate or diverge; Q estimates spiral rather than converge.

The fix is the *target network* θ⁻: a frozen snapshot of the weights used only for computing targets:

```
L(θ) = E[ (r + γ · max_{a'} Q_{θ⁻}(s', a') − Q_θ(s, a))² ]
```

θ is the network being actively trained, updated on every gradient step. θ⁻ is held fixed for a long stretch — say, 1000 gradient updates — then hard-copied from θ and frozen again. During each freeze period, the target r + γ · max Q_{θ⁻}(s', a') is a constant with respect to θ, so you're doing ordinary regression against a fixed objective. After C steps the target shifts, but discretely and slowly rather than continuously chasing its own tail. The target network doesn't change the algorithm's fixed point — it still converges to the same Q* — it just makes the path there stable enough to follow.

A complementary stabiliser is the *replay buffer*: store past transitions and sample mini-batches randomly rather than training on consecutive game steps. Consecutive transitions are highly correlated — if you just ran through the same sub-board three moves in a row, those updates all push in the same direction and overfit to the recent trajectory. Random sampling

from a large buffer breaks that correlation, so each gradient step sees a diverse mix of experiences.

Target network and replay buffer are the two engineering insights that turned DQN from a theoretically appealing idea into something that actually trains. Neither appears in the tabular Q-learning derivation; both are responses to the instabilities that emerge when you replace the table with a neural network.

For board games with large branching factors, DQN still has a structural weakness: the policy is entirely implicit — you act greedily with respect to Q_θ, with no explicit exploration mechanism, and the max over actions requires Q estimates to be accurate across all actions simultaneously before the greedy choice is reliable. But DQN moved RL out of the tabular regime, and its stabilisation tricks reappear in more sophisticated systems.

---

8. Policy Gradients and REINFORCE

A different philosophical starting point: instead of learning a value function and deriving a policy from it, directly learn the policy.

Define a parametrised policy π_θ(a|s): a neural network that takes a state and outputs a probability distribution over actions. The *goal* of training is to find the θ that maximises **J(θ)** — the expected total reward when playing full games under this policy:

```
J(θ) = E_π_θ[ G_0 ] = E_π_θ[ r_0 + γr_1 + γ²r_2 + ... ]
```

Think of J(θ) as: run a thousand games with the current policy and average the outcomes. A policy that wins more often has a higher J(θ). You can't compute J(θ) directly — that would require

averaging over all possible games — but the **policy gradient theorem** says you can estimate its gradient from samples:

```
∇_θ J(θ) = E[ ∇_θ log π_θ(a_t | s_t) · G_t ]
```

The log appears because ∇ log π = (∇π)/π — it normalises the gradient by the current probability, so the update is not dominated by actions that are already common. The mechanism is: if a game was won (G_t > 0), increase the probability of every action taken in that game; if lost (G_t < 0), decrease them. The simplest algorithm implementing this is REINFORCE.

Here G_t is the *return* — the total discounted reward from step t to the end of the game. The gradient says: if the game was won (G_t > 0), increase the probability of every action taken; if the game was lost (G_t < 0), decrease them. The simplest algorithm that implements this is called REINFORCE. REINFORCE is, in structure, the Monte Carlo approach applied to policy learning: collect the full episode, then update. The same full-game requirement, the same variance problem, applied to a direct policy rather than a value function.

The intuition is appealing. But before we get to the problems, notice something about G_t:

```
G_t = r_t + γ·r_{t+1} + ... + γ^{T-t}·r_T
```

This requires knowing every future reward out to the terminal step T. You cannot compute G_t for any step until the game is over. REINFORCE is a Monte Carlo method — you play the full episode, observe the outcome, and only then compute gradients and update. For Ultimate TTT, where every intermediate reward is zero and only the terminal state yields ±1 or −0.5, this simplifies to:

```
G_t = γ^{T-t} · r_T
```

The same discounted final outcome multiplies every step's gradient — move 1, move 23, move 39, all of them. You can't update mid-game. You can't update after move 20. You wait until the game ends, then push the gradient through all 40 steps at once.

The problem is that G_t is the game-level return, applied to every step in the trajectory. This is what the credit-assignment literature calls the "episode villain" problem: a winning game rewards even the blunders it survived. If you won a 40-move game where move 7 was a mistake, REINFORCE increases the probability of move 7, because G_7 > 0. The same G_t multiplies the gradient of every action in the sequence — good and bad alike. This is maximal-variance credit assignment.

For short sequences — a few moves — REINFORCE can work. For a 40-move game with long-range dependencies, the noise overwhelms the signal. The algorithm technically converges given enough data, but "enough" turns out to be an impractical quantity.

---

9. Actor-Critic: A Partial Fix

The actor-critic architecture is a hybrid that addresses the variance problem in policy gradients. Learn both a policy — the **actor**, π_θ(a|s) — and a value function — the **critic**, V_φ(s). Use the critic to reduce variance in the policy gradient. Actor-critic is, in structure, the TD approach applied to policy learning: update after each step using a one-step bootstrap, rather than waiting for the game to end. The same tension between bias and variance, now applied to the policy gradient.

Specifically, instead of multiplying by the full return G_t, multiply by the **advantage**:

```
Â_t = Q(s_t, a_t) − V(s_t)
```

The advantage asks: was this action better than what you'd typically get from this state? Not "did the game end well?" but "was this specific move above or below the baseline?" The policy gradient update becomes:

```
θ ← θ + α · ∇_θ log π_θ(a_t | s_t) · Â_t
```

Q(s_t, a_t) isn't directly available, but you can estimate it with a one-step TD target:

```
Q(s_t, a_t) ≈ r_t + γ · V_φ(s_{t+1})
```

Substitute that in:

```
Â_t ≈ r_t + γ · V_φ(s_{t+1}) − V_φ(s_t) = δ_t
```

The advantage *is* the TD error. The same quantity δ_t from Section 5 serves double duty: the critic minimises it as a loss, and the actor uses it as its gradient weight. Critically, you can compute δ_t after a single step — you just need r_t, s_{t+1}, and the critic's current estimates. No waiting for the game to end. This is fully online: observe (s_t, a_t, r_t, s_{t+1}), compute δ_t, update both actor and critic, move on. This is the key improvement over REINFORCE.

This is genuinely better. The advantage is a local signal — it measures the quality of one action relative to the state it's taken from. The variance in the gradient estimate drops substantially. Actor-critic methods (and their descendant algorithms like PPO

and A3C) form the backbone of modern deep RL for continuous control, robotics, and many strategy games.

REINFORCE vs Actor-Critic vs GAE

Method	Needs full game?	Update frequency	Variance	Bias
REINFORCE	Yes	Per episode	High	Low
Actor-Critic (1-step)	No	Per step	Lower	Higher
GAE/PPO	No (fixed rollout)	Per rollout	Tunable (λ)	Tunable (λ)

Generalised Advantage Estimation (GAE) is the practical solution to the bias-variance tradeoff. Instead of committing to either 1-step TD (high bias, low variance) or full Monte Carlo (low bias, high variance), it blends them continuously with a parameter λ:

```
Â_t^GAE = δ_t + (γλ)δ_{t+1} + (γλ)²δ_{t+2} + ...
```

where each δ_{t+l} = r_{t+l} + γV(s_{t+l+1}) − V(s_{t+l}) is the TD error at step t+l. Setting λ=0 gives pure 1-step TD (only δ_t survives). Setting λ=1 recovers the full Monte Carlo return. In practice λ ≈ 0.95 sits in the sweet spot: low enough variance to be stable, high enough horizon to capture meaningful credit. You collect a short fixed-length rollout (not a full game), compute GAE over it, then update.

Proximal Policy Optimisation (PPO) solves a different failure mode: vanilla policy gradients can collapse. If a gradient step

accidentally makes the policy much worse — sending it to a low-reward region — it becomes unlikely to take the actions that would recover, so it never learns its way back. PPO prevents this by clipping the update: you are not allowed to change the policy more than ε in any single step.

The clipped objective is:

```
L^CLIP(θ) = E[ min( r_t(θ) · Â_t,  clip(r_t(θ), 1-ε, 1+ε) · Â_t
```

where r_t(θ) = π_θ(a_t|s_t) / π_θ_old(a_t|s_t) is the ratio of the new policy's probability to the old policy's probability for the action actually taken. If the ratio is inside [1-ε, 1+ε], the gradient flows normally. If the policy tries to drift outside that band, the gradient is clipped to zero — the update stops. This enforces a *trust region*: each gradient step is bounded in how much it can change the policy, keeping training stable regardless of noise in the advantage estimates.

PPO has become the dominant practical algorithm for deep RL. The combination of GAE (good advantage estimates) and PPO (stable updates) is what made policy gradient methods reliable enough to deploy at scale.

Reinforcement Learning from Human Feedback (RLHF) is the most prominent application of PPO outside of games. Large language models (LLMs) like GPT-4 or Claude are first trained by predicting the next word across enormous text corpora — this gives them language ability but doesn't directly optimise for being *helpful* or *safe*. RLHF adds a second training phase using RL.

The setup: human raters compare pairs of model responses and indicate which is better. These preferences train a *reward model* — a separate neural network that scores any given response. Then PPO is used to fine-tune the LLM to maximise the reward

model's score. The LLM is the policy (π_θ), the conversation so far is the state, generating each token is an action, and the reward comes from the reward model at the end of the response.

PPO is used specifically because its clipping constraint prevents the model from drifting too far from the pretrained weights — a useful property when you've spent enormous compute on pretraining and don't want a single RL fine-tuning step to destroy it. The policy gradient framework, originally developed for games, turned out to be exactly what was needed for aligning language models with human preferences.

But there's a fundamental ceiling here. The advantage estimate is only as good as V_φ, and V_φ is only as good as the training data you've generated so far. For a game with 40-move trajectories and complex long-range dependencies, even a well-trained critic makes noisy estimates. You're still, at root, trying to attribute a game-level outcome back to individual moves via an imperfect intermediary function.

---

10. The Remaining Problem

Let's be precise about why this matters for Ultimate TTT.

When your opponent sends you to a particular sub-board, the quality of that decision — their decision — may not be reflected in the game outcome for another 10 or 15 moves. In the meantime, many other things happen. Your value function has to absorb this uncertainty and still provide a useful per-step signal.

The problem compounds during training. Early in self-play, both players are weak. The games are noisy, wins and losses are partially random, and the value function is learning from this garbage data. As the network improves, the value function

improves, but it's always lagging behind the policy, and the credit-assignment signal is always contaminated by the imprecision of a function trained on earlier, weaker play.

Actor-critic methods work well for many RL problems. They fail to achieve superhuman performance on complex board games, not because the algorithm is wrong, but because the credit-assignment problem at 40-move horizon is too hard for a learned value function to solve accurately enough.

This isn't a fixable implementation detail. It's a structural limitation of trajectory-level credit assignment.

The answer requires a different framing entirely. Before we get there, we need to understand one more idea: how to explore intelligently when you don't know which options are worth investigating. That's the subject of the next piece.

The Landscape at a Glance

The table below situates each approach by the training signal it uses and how local that signal is. "Local" means closer to the individual decision; "global" means the signal derives from the game-level outcome.

Method	Training Signal	Locality	Core Weakness
Monte Carlo	Full return G	Global (whole game)	High variance; ignores step-level information
TD Learning	δ_t (one-step Bellman error)	Per step	Requires good value function bootstrap

Method	Training Signal	Locality	Core Weakness
Q-Learning	Bellman optimality update	Per (state, action)	Doesn't scale to large state spaces
DQN	Q_θ Bellman update	Per (state, action)	Needs accurate Q over all actions; implicit policy
REINFORCE	G_t multiplied per action	Global (whole episode)	"Episode villain" — rewards blunders in winning games
Actor-Critic	Advantage $\hat{A}_t = Q - V$	Per step (approx.)	Quality ceiling: bounded by critic accuracy

Notice the direction of travel: from global to local. Each successive method tries to get a sharper signal closer to the individual decision. Actor-critic methods are the ceiling of trajectory-level attribution — and as the last section showed, that ceiling isn't high enough for complex board games.

This series documents the construction of an Ultimate Tic-Tac-Toe AI using AlphaZero-style MCTS and self-play. The code is open source. Earlier: [Why Ultimate TTT is a Surprisingly Hard Game]. Next: [essay1c: Bandits, Exploration, and the Bridge to MCTS].

Bandits, Exploration, and the Bridge to MCTS

Essay 1b established why trajectory-level credit assignment fails for complex games. Every method we looked at — from TD learning through actor-critic — is trying to attribute a game-level outcome back to individual moves, and that attribution gets noisier the longer the game. Before we see the solution, we need one more idea: intelligent exploration.

---

1. The Exploration Problem: Bandit Algorithms

Before scaling up to full games, there's a clean version of the exploration problem worth understanding on its own: the *multi-armed bandit*.

You're sitting in front of K slot machines, each with an unknown reward distribution. You have N total pulls to spend. Your goal is to maximise total reward. The tension is immediate: pull an arm you've tried before and you're exploiting your current estimates; pull an arm you haven't tried much and you're exploring in hopes of finding something better. Pull too greedily and you miss the best arm. Pull too randomly and you waste pulls on bad ones.

The standard solution is *Upper Confidence Bound (UCB)*. For each arm a, maintain an empirical mean Q(a) and a count N(a) of how many times you've pulled it. Always pull the arm with the highest UCB score:

```
UCB(a) = Q(a) + c · √(log(N) / N(a))
```

The first term is exploitation: pull the arm that's been good so far. The second term is exploration: the bonus is large when an arm has been pulled rarely (small N(a)) and shrinks as the arm gets tried more. c controls the balance. The elegant property is that UCB is *optimistic under uncertainty* — it assumes under-explored arms might be better than they look, and only stops exploring them once you have enough evidence that they're not.

This isn't a heuristic. UCB has a theoretical guarantee: it accumulates regret (total reward lost relative to always pulling the best arm) at a rate that grows only logarithmically in N. You'll fall behind the oracle policy, but the gap closes as you gather more data. The exploration bonus is precisely calibrated to ensure this.

---

2. PUCT: UCB with a Neural Prior

MCTS uses a direct descendant of UCB called PUCT. The setup is the same — from a node in the search tree, you must choose which child to visit — but there's a key difference: the neural network already has beliefs about which moves are promising, encoded in the policy prior P(s, a). The PUCT formula is:

```
PUCT(s, a) = Q(s, a) + c_puct · P(s, a) · √N(s) / (1 + N(s, a))
```

The exploitation term Q(s, a) is the mean backed-up value across all simulations that have passed through the move (s, a) so far. The exploration term P(s, a) · √N(s) / (1 + N(s, a)) is large when the move has been tried rarely relative to the total visits at this node, weighted by the network's prior probability for that move.

The AlphaGo innovation was adding P(s, a) as a weight on the exploration bonus. In a pure bandit, every arm starts equal and earns exploration in proportion only to how undersampled it is. In MCTS, the policy network already has informed guesses — so well-regarded moves get explored first, rather than all moves being treated as equally unknown. Weak moves need to earn attention by proving themselves; strong moves get a head start. This is especially important in games with large branching factors: spreading exploration budget evenly across 80 legal moves wastes most of it.

This also explains how r, Q, and V connect inside search. The environment's r only enters at terminal nodes — when a simulation reaches the end of the game and the actual ±1 outcome is returned. At every non-terminal leaf, $V_\theta(s)$ — the trained network's value head — evaluates the position instead of simulating to the end. Q(s, a) inside the tree is then the running mean of V_θ across all simulations that passed through that edge. So PUCT never sees raw r directly during search: r shaped V_θ during training, and V_θ now drives Q inside the tree. They operate at different levels but are tightly connected through the value network.

To make the chain explicit: the game's reward function r is the ultimate source of truth; V_θ is the compressed representation of it; Q is the simulation-averaged version of V_θ; and PUCT uses Q to decide where to look next.

---

3. From Bandits to Trees

The bandit problem is selecting one action from a list. Choose an arm, observe a reward, update your estimates, repeat. The state doesn't change. There's no tomorrow — each pull is independent.

A game isn't like that. You're selecting a *sequence* of actions, each taken from a state that depends on all prior choices. The decision at move 7 shapes what decisions are even available at move 15. You can't evaluate a move in isolation; you have to evaluate it in context.

This is where MCTS extends UCB from a list into a tree. The key insight is simple: apply the same explore/exploit logic at every node.

Each position in the game is a bandit problem. From that position, you have a set of legal moves. Each move is an arm. You don't know which arm is best — that depends on what happens afterward, which depends on your opponent's response, which depends on your response to that, and so on. So you run simulations. Each simulation is a sequence of PUCT selections, one at each position you encounter, until you reach a position you haven't visited before.

At that leaf, you evaluate with the network (or simulate to the end if you're doing rollouts). Then you back the value up through every node on the path. Each node's Q(s, a) gets updated — the mean value across all simulations through that edge. On the next simulation, PUCT at each node sees updated Q values and chooses again.

Run this 800 times and you've built a miniature game tree, weighted toward the promising branches. Nodes near the top have been visited hundreds of times; deep, unpromising branches may have been visited once. The visit distribution isn't uniform — it's been shaped by the same optimism-under-uncertainty principle as UCB, applied recursively through the tree.

The bandit formulation gives you the right unit of reasoning: not "what's the best game to play from here?" (unknowable) but

"which move at *this node* deserves more simulation budget?" That question is answerable, and answering it 800 times gives you something much better than a single network forward pass.

---

4. Why This Changes the Credit-Assignment Picture

Essay 1b ended with a structural ceiling: actor-critic methods can't attribute game outcomes to individual moves accurately enough for complex games. MCTS doesn't try.

Instead of asking "which move caused the win?", MCTS asks "after 800 simulations from this position, which moves were actually visited?" The visit distribution — call it π_MCTS — is a better policy than the raw network output. If the network assigned 30% probability to move A but it was only visited 5% of the time (because every follow-up was bad), the tree has corrected the prior. π_MCTS is the policy after looking ahead, not the policy from pattern-matching alone.

This distribution becomes the training target. The network learns to predict π_MCTS, not the game outcome directly. Every position generates a local, high-quality signal: not "did the game end well?" but "after careful search from here, which moves are actually good?" The credit-assignment problem doesn't disappear, but it's been replaced with something much more tractable.

Now we're ready to see how MCTS actually works — the four phases (Select, Expand, Evaluate, Backup), how the tree is built incrementally across simulations, and how the visit distribution flows back as a training signal into the network.

This series documents the construction of an Ultimate Tic-Tac-Toe AI using AlphaZero-style MCTS and self-play. The code is open source. Earlier: [essay1b: The Credit-Assignment Problem]. Next: [Monte Carlo Tree Search: How to Think When You Can't Calculate].